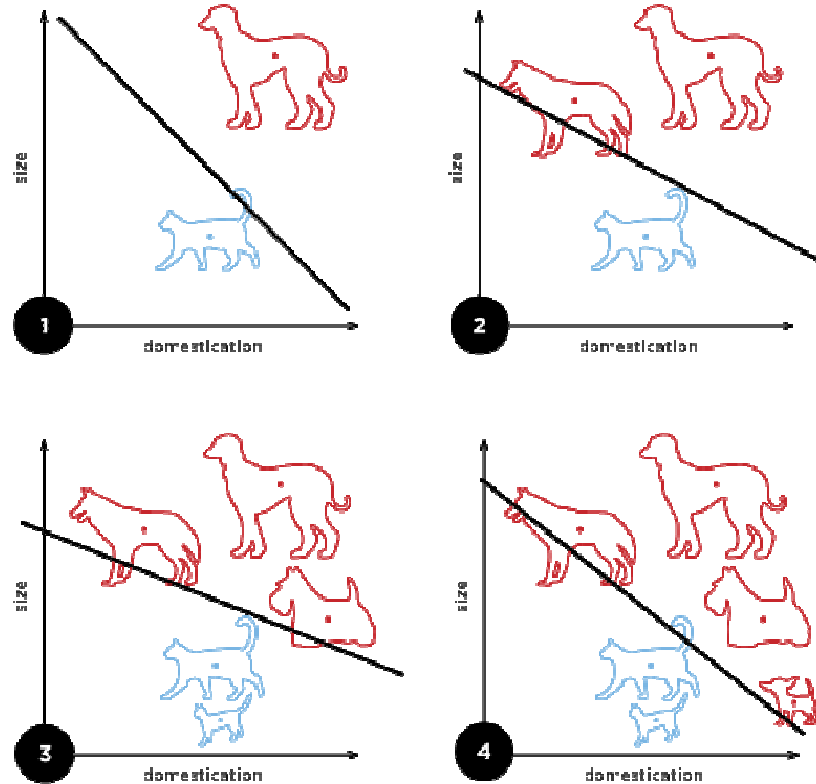
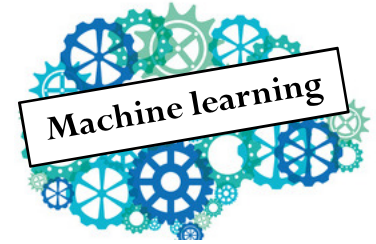


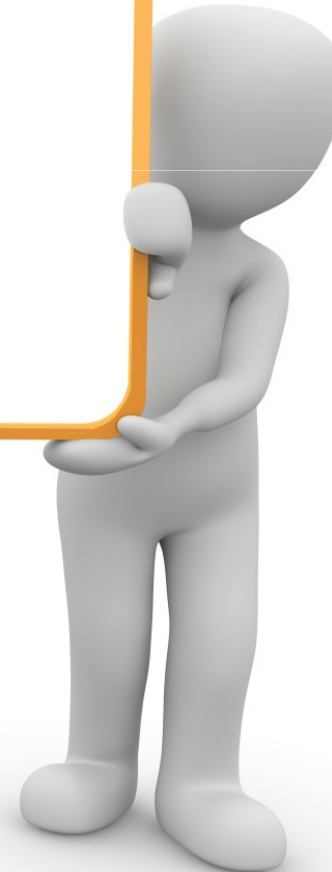
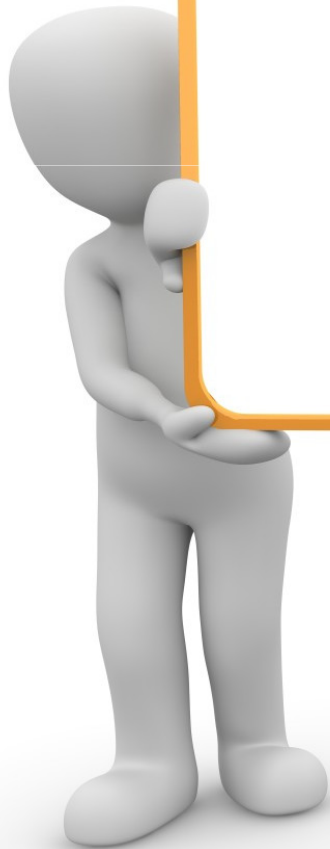


Autres algorithmes de classification supervisée

Plan

2

- KNN
- Perceptron
- Classification naïve bayésienne



3

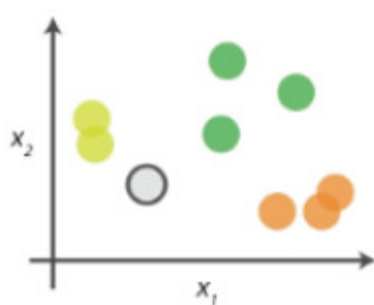
KNN

Algorithme K plus proches voisins

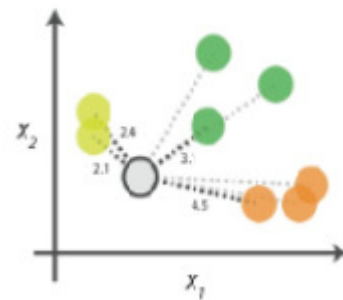
4

□ **Idée:** étant donnée une entrée X

1. trouver les k entrées X_t parmi les exemples d'apprentissage qui sont les plus « proches » de X
2. faire voter chacune de ces entrées pour leur classe associée y_t
3. retourner la classe majoritaire



Classer le point gris dans une classe



Calculer la distance entre le point gris et les autres points

Point Distance			
●	●	2.1	→ 1st NN
●	●	2.4	→ 2nd NN
●	●	3.1	→ 3rd NN
●	●	4.5	→ 4th NN

Trouver les voisins les plus proches

Class	# of votes	
●	2	→ Class ● wins the vote! Point ● is therefore predicted to be of class ●.
●	1	
●	1	

Voter sur les étiquettes de classes prévues en fonction des classes des k voisins les plus proches

Algorithme K plus proches voisins

5

- Le succès de cet algorithme va dépendre de deux facteurs
 - ▣ la quantité de données d'entraînement
 - plus il y en a, meilleure sera la performance
 - ▣ la qualité de la mesure de distance (est-ce que deux entrées jugées similaires sont de la même classe?)
 - en pratique, on utilise souvent la distance Euclidienne:

$$d(\mathbf{x}_1, \mathbf{x}_2) = \sqrt{\sum_k (x_{1,k} - x_{2,k})^2}$$

$\mathbf{X}$ = vecteur
 x = scalaire

Illustration : 3 plus proches voisins

6

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?

Ensemble d'entraînement

(100 exemples d'apprentissage par classe)



Illustration : 3 plus proches voisins

7

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?

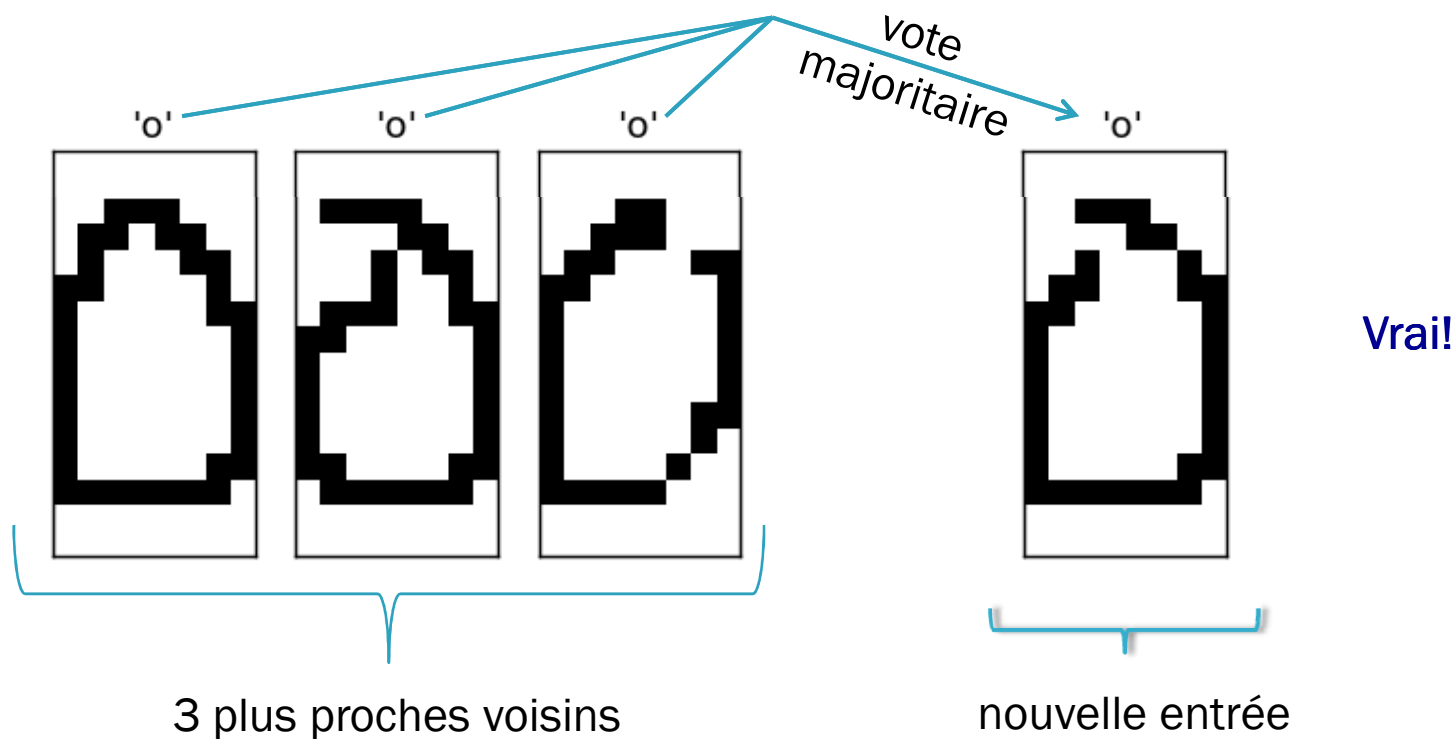


Illustration : 3 plus proches voisins

8

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?

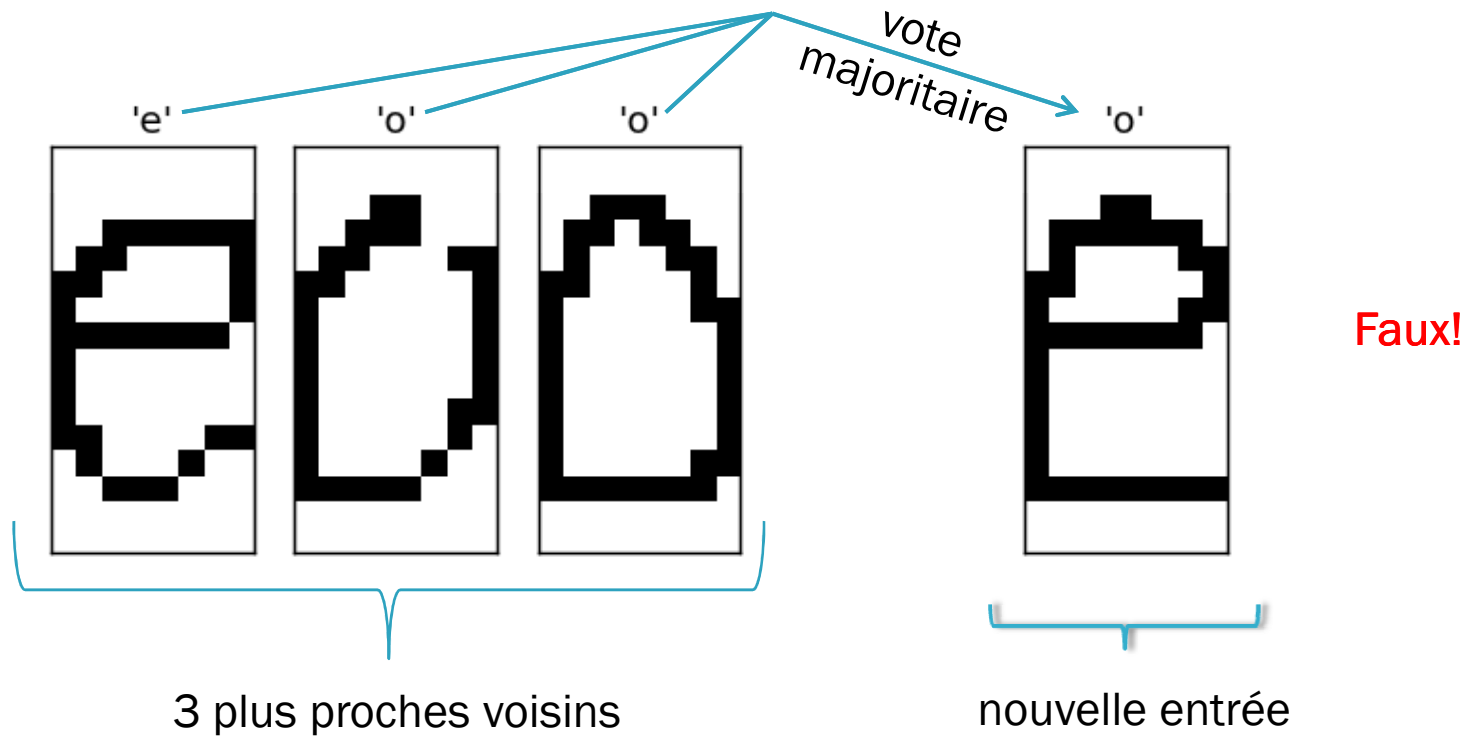
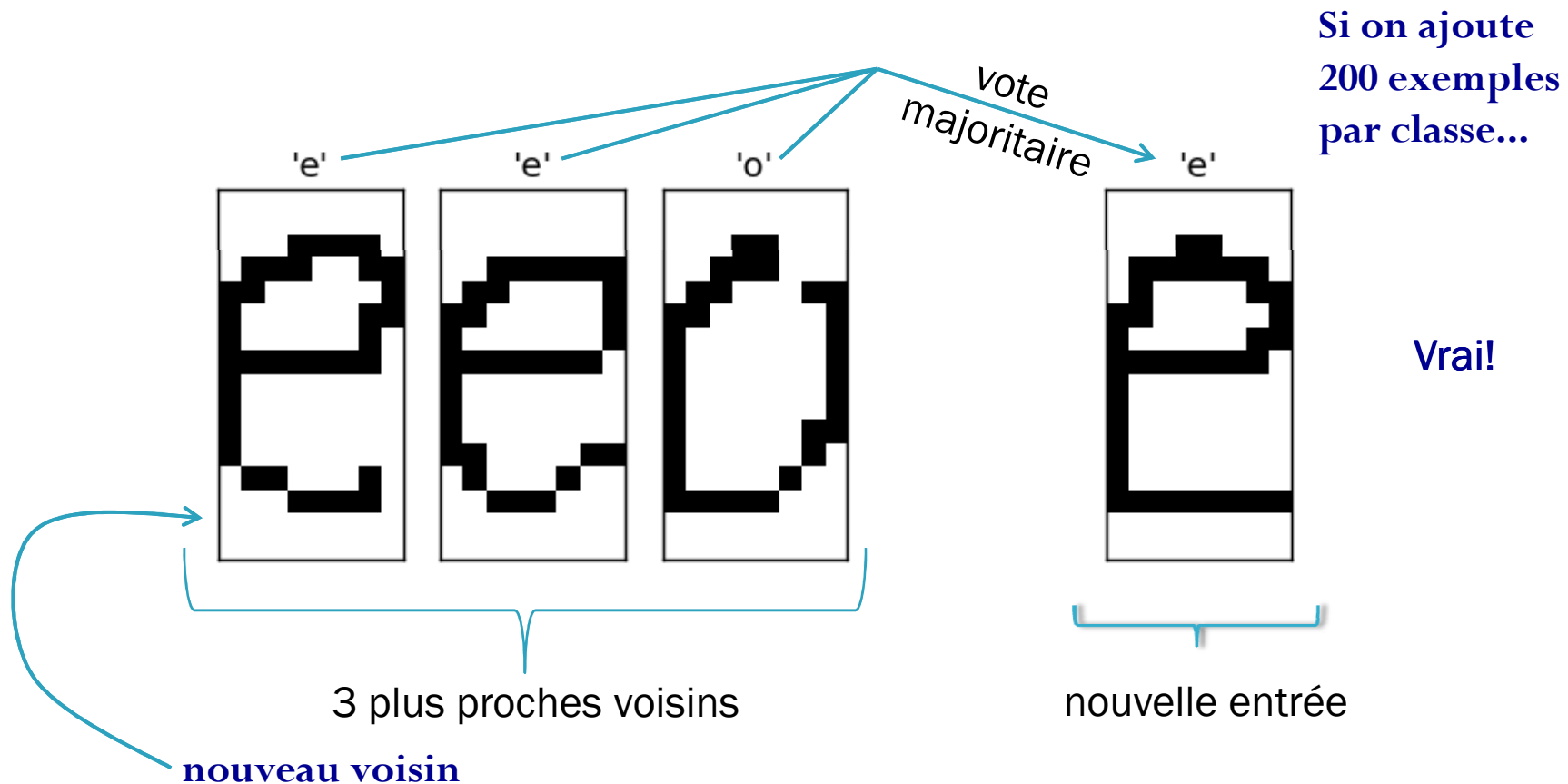


Illustration : 3 plus proches voisins

9

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?



KNN

Exercice

10

Soit le jeu de données suivant :

Name	Acid durability	Strength	Class
Type_1	7	7	Bad
Type_2	7	4	Bad
Type_3	3	4	Good
Type_4	1	4	Good

Nous voulons prédire la classe de l'instance $\langle \text{acid durability} = 3, \text{strength} = 7 \rangle$ pour :

- $K = 1$
- $K = 3$

KNN

Exercice - correction

11

Calcul de la distance :

Name	Acid durability	Strength	Class	Distance
Type_1	7	7	Bad	4
Type_2	7	4	Bad	5
Type_3	3	4	Good	3
Type_4	1	4	Good	3.6

Classement selon le voisin le plus proche :

Name	Acid durability	Strength	Class	Distance	Rang
Type_1	7	7	Bad	4	3
Type_2	7	4	Bad	5	4
Type_3	3	4	Good	3	1
Type_4	1	4	Good	3.6	2

KNN

Exercice - correction

12

K = 1 : classe = Good

Name	Acid durability	Strength	Class	Distance	Rang
Type_1	7	7	Bad	4	3
Type_2	7	4	Bad	5	4
Type_3	3	4	Good	3	1
Type_4	1	4	Good	3.6	2

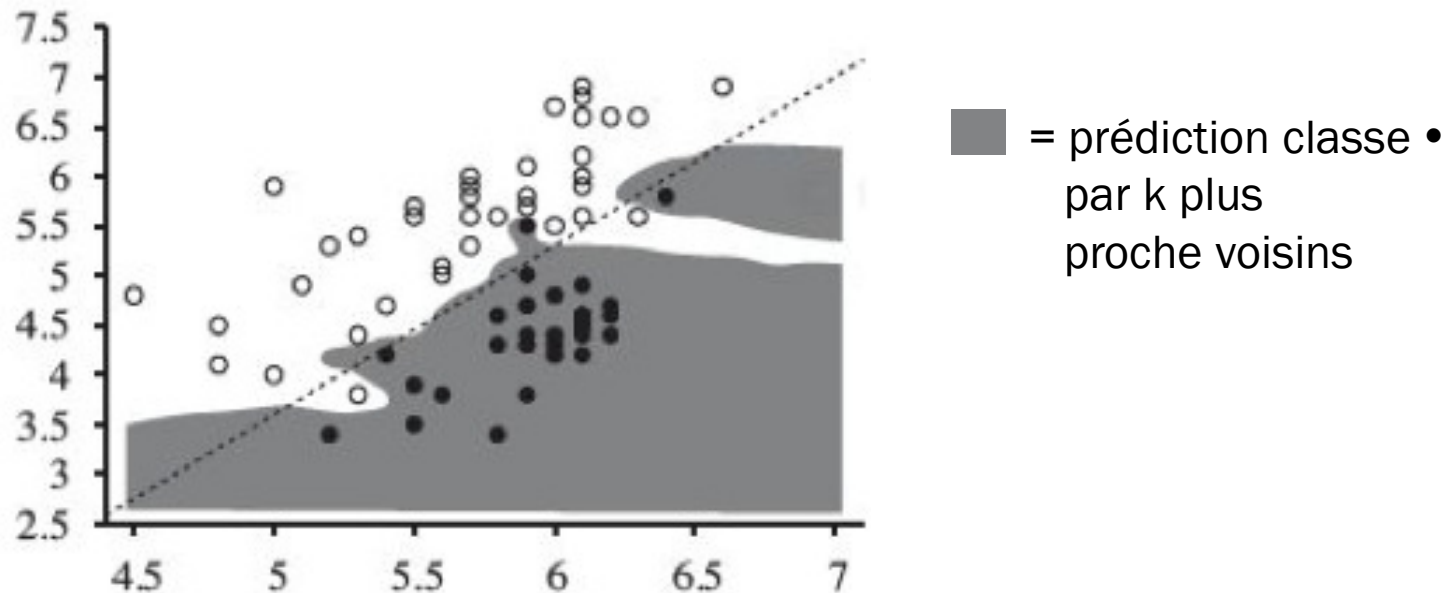
K = 3 : classe = Good

Name	Acid durability	Strength	Class	Distance	Rang
Type_1	7	7	Bad	4	3
Type_2	7	4	Bad	5	4
Type_3	3	4	Good	3	1
Type_4	1	4	Good	3.6	2

Avantages et limites

13

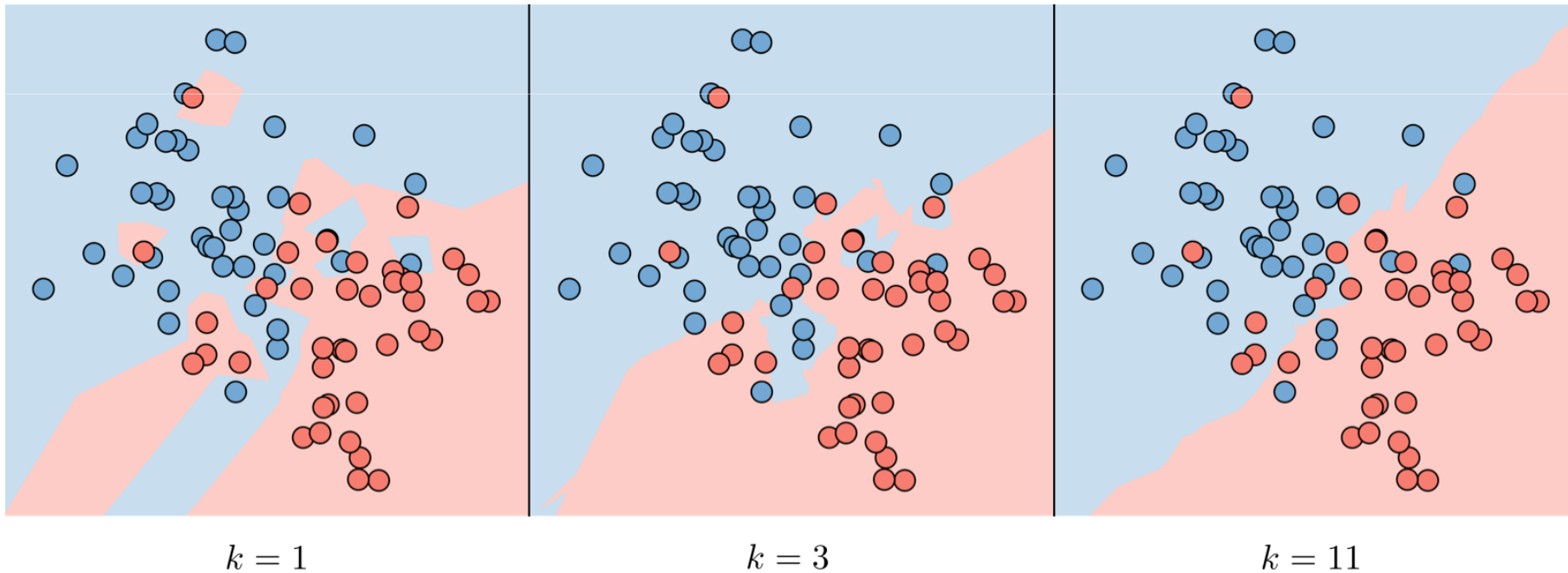
- k plus proche voisins est non-linéaire
 - ▣ mais coûteux en mémoire et temps de calcul
 - pas approprié pour des problèmes avec beaucoup de données



Choix de k

14

- Plus le paramètre k est élevé, plus le biais est élevé, et plus le paramètre k est faible, plus la variance est élevée

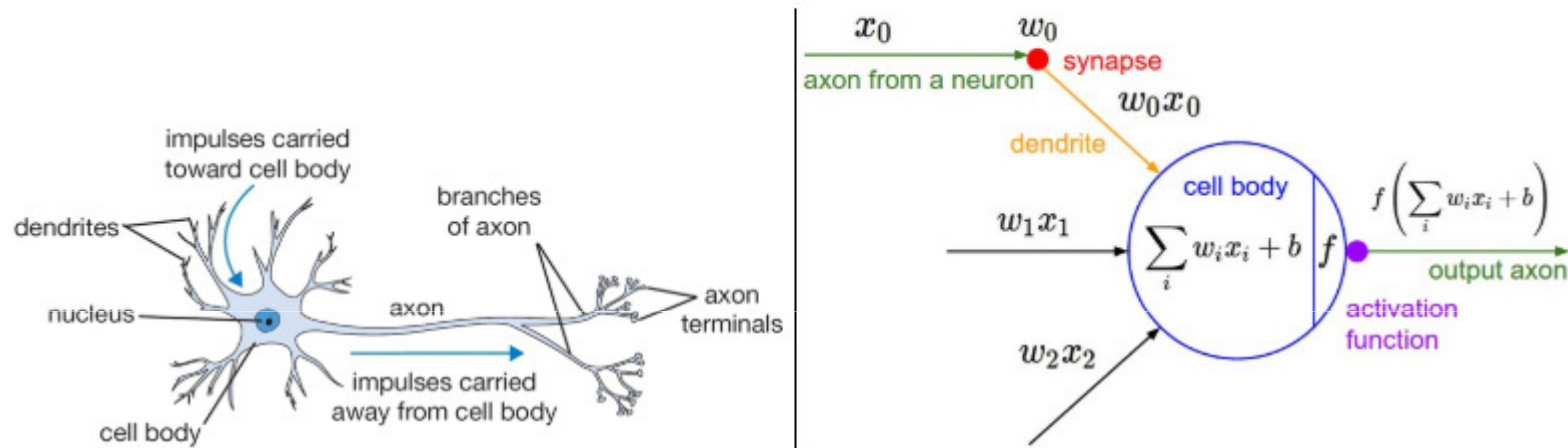


15

Perceptron

Neurone et réseaux de neurones

16



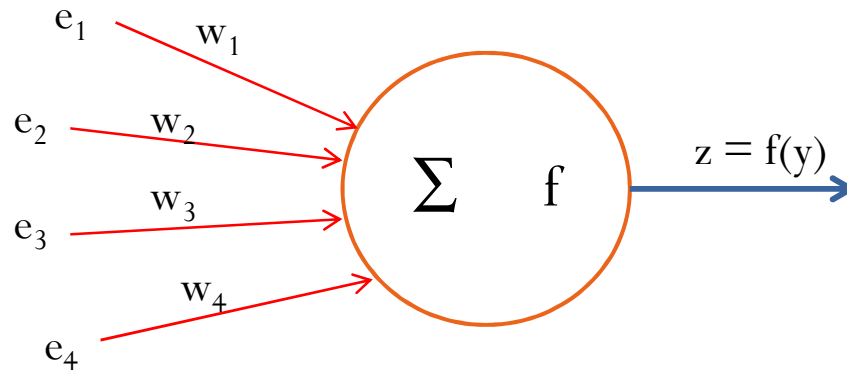
A cartoon drawing of a biological neuron (left) and its mathematical model (right).

Synapse : le point de contact entre un neurone et un autre neurone ou un autre type de cellule (ex : cellule musculaire)

Le neurone reçoit des signaux via les dendrites et émet un signal via l'axone. La communication entre les neurones se fait via les synapses.

Neurone artificiel

17



$$y = w_1 \cdot e_1 + w_2 \cdot e_2 + w_3 \cdot e_3 + w_4 \cdot e_4$$

e_i : entrées du neurone

w_i : coefficients synaptiques (poids)

Σ : somme pondérée des entrées via les poids

f : fonction seuil

Le Perceptron

18

□ **Idée:** modéliser la décision à l'aide d'une fonction linéaire, suivi d'un seuil:

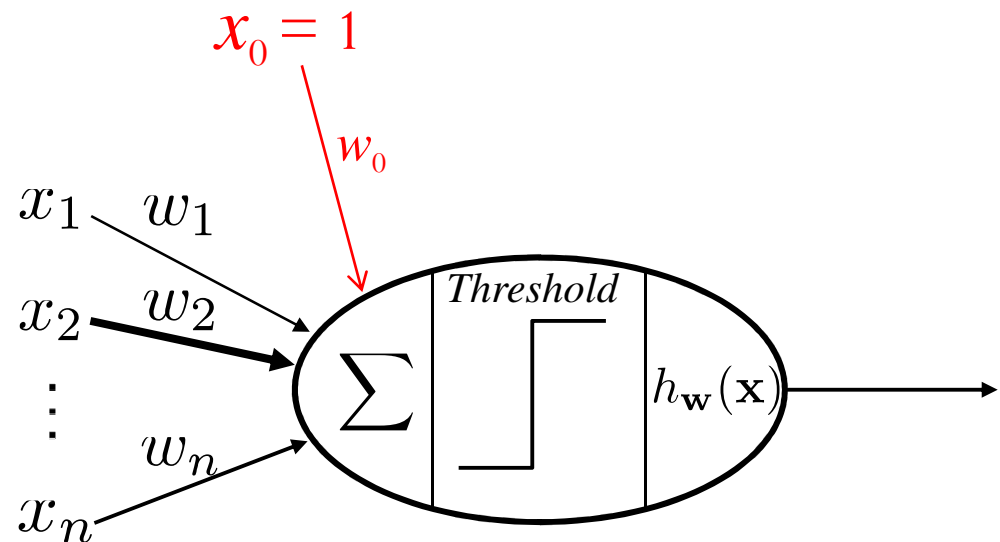
□ $h_{\mathbf{w}}(\mathbf{x}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x})$

où $\text{Threshold}(z) = 1$ si $z > 0$, $\text{Threshold}(z) = 0$ sinon

fonction de Heaviside

Le **vecteur de poids** $\mathbf{W}$ correspond aux **paramètres** du modèle.

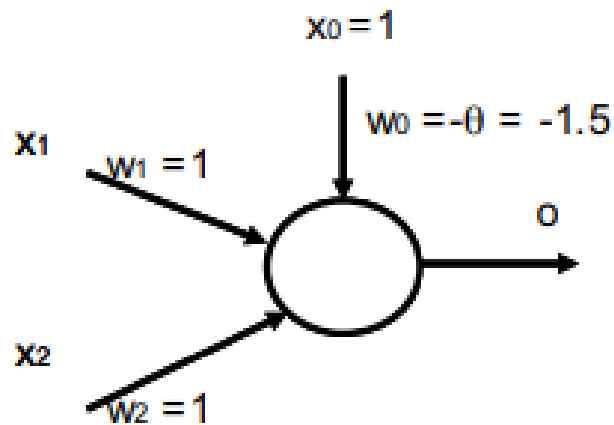
On ajoute également un biais w_0 , qui équivaut à ajouter une entrée fictive $x_0 = 1$



Exemple 1

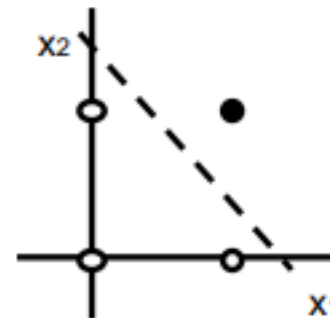
19

□ ET logique



x_1	x_2	o
0	0	0
0	1	0
1	0	0
1	1	1

AND

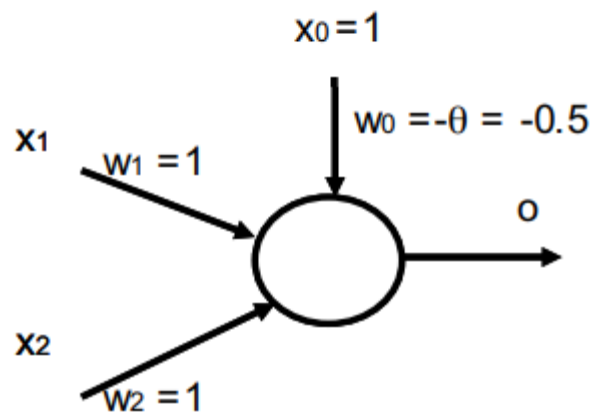


$$x_2 = -\frac{w_1}{w_2}x_1 + \frac{\theta}{w_2}$$

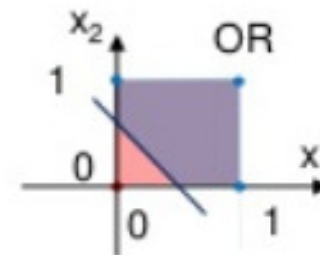
Exemple 2

20

□ OU logique



x_1	x_2	o
0	0	0
0	1	1
1	0	1
1	1	1

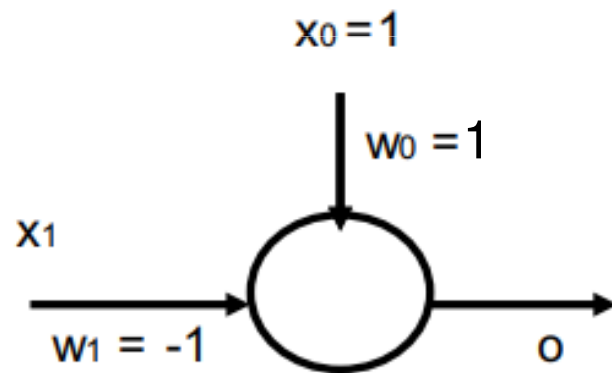


Perceptron

Exemple 3

21

□ NOT



x_1	o
0	1
1	0

Algorithme du Perceptron

22

- L'algorithme d'apprentissage doit adapter la valeur des paramètres (les poids et le biais) de façon à ce que $h_w(x)$ soit la bonne réponse sur les données d'entraînement

□ Algorithme du Perceptron

1. pour chaque paire $(x^{(t)}, y^{(t)}) \in D$
 - a. Calculer $h_w(x^{(t)}) = \text{Threshold}(w \cdot x^{(t)})$
 - b. Si $y^{(t)} \neq h_w(x^{(t)})$
 - $w_i \leftarrow w_i + \alpha (y^{(t)} - h_w(x^{(t)})) x_i^{(t)} \quad \forall i$ (mise à jour des poids et biais)
2. retourner à 1 jusqu'à l'atteinte d'un critère d'arrêt (nb. maximal d'itérations atteint ou nb. d'erreurs est 0)

- La mise à jour des poids est appelée la **règle d'apprentissage du Perceptron**.
- Le multiplicateur α est appelé le **taux d'apprentissage**

Algorithme du Perceptron

23

- L'algorithme d'apprentissage doit adapter la valeur des paramètres (les poids et le biais) de façon à ce que $h_w(x)$ soit la bonne réponse sur les données d'entraînement

□ Algorithme du Perceptron

1. pour chaque paire $(x^{(t)}, y^{(t)}) \in D$
 - a. Calculer $h_w(x^{(t)}) = \text{Threshold}(w \cdot x^{(t)})$
 - b. Si $y^{(t)} \neq h_w(x^{(t)})$
 - $W \leftarrow W + \alpha (y^{(t)} - h_w(x^{(t)})) x^{(t)}$ (mise à jour des poids et biais) 
2. retourner à 1 jusqu'à l'atteinte d'un critère d'arrêt (nb. maximal d'itérations atteint ou nb. d'erreurs est 0)

- La mise à jour des poids est appelée la **règle d'apprentissage du Perceptron**.
- Le multiplicateur α est appelé le **taux d'apprentissage**

Exercice

24

- Simulation avec biais, $\alpha = 0.1$
- Initialisation : $\mathbf{w} \leftarrow [0.5, 0, 0]$
 biais

Paire $(\mathbf{x}^{(1)}, y^{(1)})$:

$$h(\mathbf{x}^{(1)}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}^{(1)}) = \text{Threshold}(0.5) = 1$$

puisque $h(\mathbf{x}^{(1)}) = y^{(1)}$, on ne fait pas de mise à jour de $\mathbf{w}$

Ensemble d'entraînement	
x	y
[1,2,0]	1
[1,0,3]	0
[1,3,0]	0
[1,1,1]	1

On ajoute 1 pour le biais

Exercice

25

□ Simulation **avec biais**, $\alpha = 0.1$

□ Initialisation : $\mathbf{w} \leftarrow [0.5, 0, 0]$

D Ensemble
d'entraînement

x	y
[1,2,0]	1
[1,0,3]	0
[1,3,0]	0
[1,1,1]	1

Paire $(\mathbf{x}^{(2)}, y^{(2)})$:

$$h(\mathbf{x}^{(2)}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}^{(2)}) = \text{Threshold}(0.5) = 1$$

puisque $h(\mathbf{x}^{(2)}) \neq y^{(2)}$, on met à jour $\mathbf{w}$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha (y^{(2)} - h(\mathbf{x}^{(2)})) \mathbf{x}^{(2)} = [0.5, 0, 0] + 0.1 * (0 - 1) [1, 0, 3] = [0.4, 0, -0.3]$$

Exercice

26

- Simulation **avec biais**, $\alpha = 0.1$
- Initialisation : $\mathbf{w} \leftarrow [0.4, 0, -0.3]$

D Ensemble
d'entraînement

x	y
[1,2,0]	1
[1,0,3]	0
[1,3,0]	0
[1,1,1]	1

Paire $(\mathbf{x}^{(3)}, y^{(3)})$:

$$h(\mathbf{x}_3) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}^{(3)} + b) = \text{Threshold}(0.4) = 1$$

puisque $h(\mathbf{x}^{(3)}) \neq y^{(3)}$, on met à jour $\mathbf{w}$ et b

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} + \alpha (y^{(3)} - h(\mathbf{x}^{(3)})) \mathbf{x}_3 = [0.4, 0, -0.3] + 0.1 * (0 - 1) [1, 3, 0] \\ &= [0.3, -0.3, -0.3]\end{aligned}$$

Exercice

27

□ Simulation **avec biais**, $\alpha = 0.1$

□ Initialisation : $\mathbf{w} \leftarrow [0.3, -0.3, -0.3]$

D Ensemble
d'entraînement

x	y
[1,2,0]	1
[1,0,3]	0
[1,3,0]	0
[1,1,1]	1

Paire $(\mathbf{x}^{(4)}, y^{(4)})$:

$$h(\mathbf{x}^{(4)}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}^{(4)} + b) = \text{Threshold}(-0.3) = 0$$

puisque $h(\mathbf{x}^{(4)}) \neq y^{(4)}$, on met à jour $\mathbf{w}$ et b

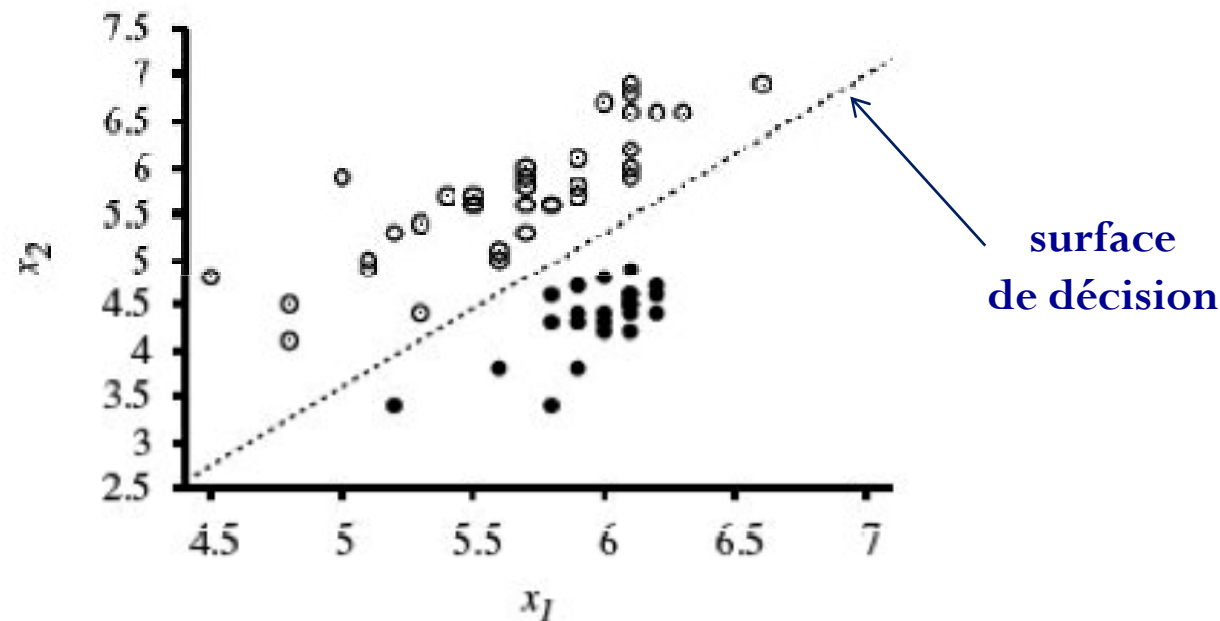
$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} + \alpha (y^{(4)} - h(\mathbf{x}^{(4)})) \mathbf{x}^{(4)} = [0.3, -0.3, -0.3] + 0.1 * (1 - 0) [1, 1, 1] \\ &= [0.4, -0.2, -0.2]\end{aligned}$$

Et ainsi de suite, jusqu'à l'atteinte d'un critère d'arrêt...

Surface de séparation

28

- Le Perceptron cherche un **séparateur linéaire** entre les deux classes

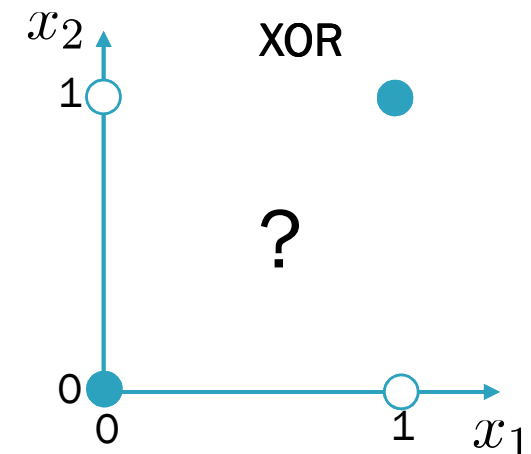
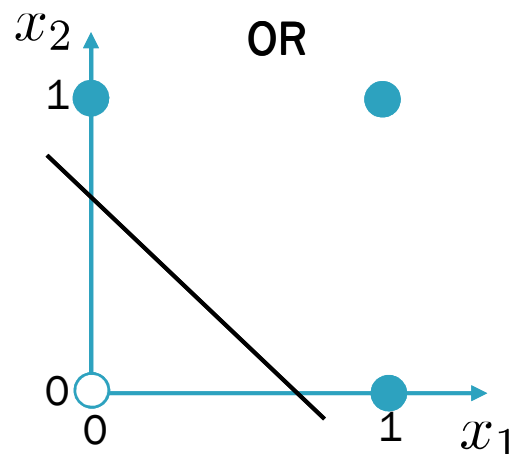
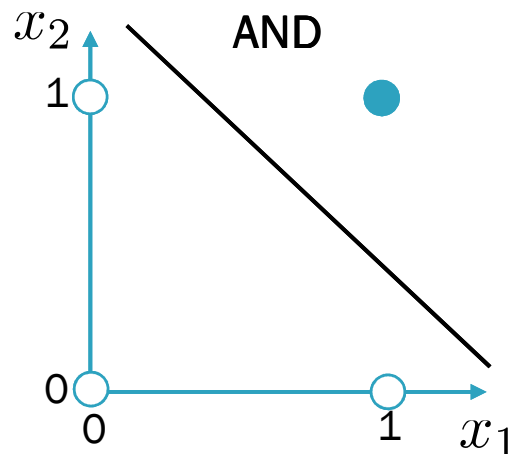


- La **surface de décision** d'un classifieur est la surface (dans le cas du perceptron en 2D, une droite) qui sépare les deux régions classifiées dans les deux classes différentes

Limitations des classifieurs linéaires

29

- Si les données d'entraînement sont séparables linéairement, le Perceptron va trouver cette séparation
- Cependant, la majorité des problèmes de classification ne sont pas linéaires
 - le perceptron simple ne peut résoudre que des problèmes linéairement séparables
 - Pour aller plus loin, il est nécessaire d'ajouter des couches

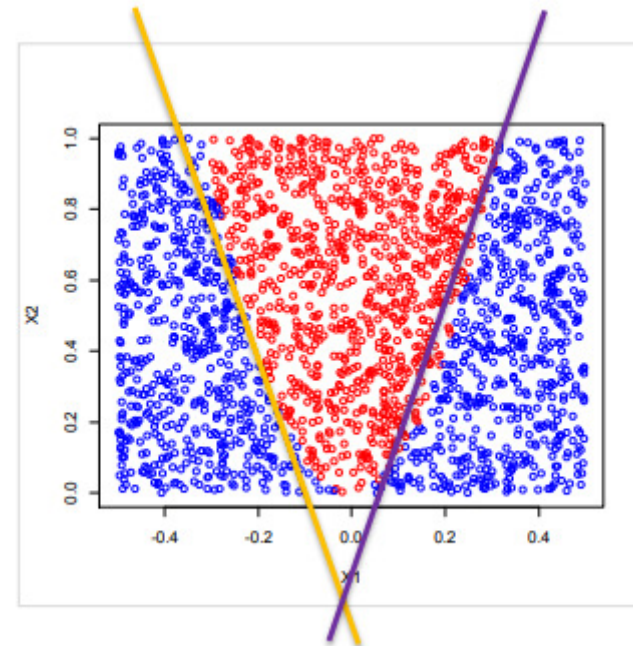
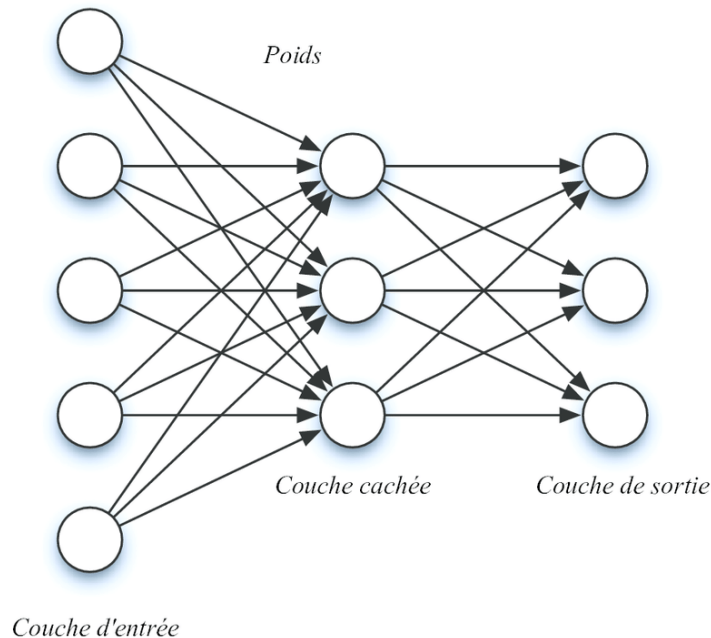


Perceptron multicouche (PMC)

Présentation

30

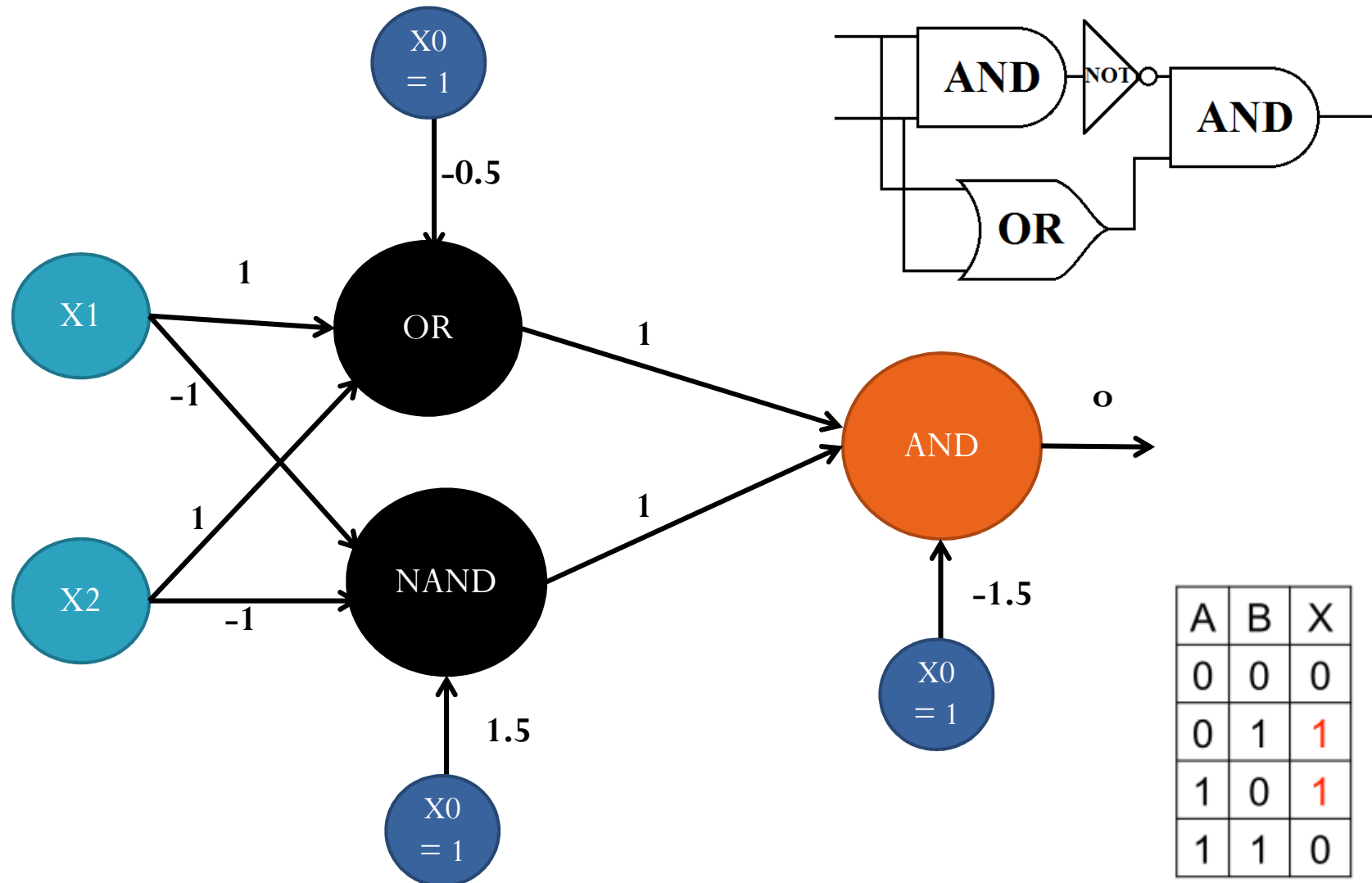
- En 1986, est inventé le perceptron multi-couches (MLP pour Multi-Layer Perceptron) avec l'algorithme de rétro propagation du gradient de l'erreur (1974, Paul Werbos et 1985, Yann Le Cun)
- Une combinaison de séparateurs linéaires permet de produire un séparateur global non-linéaire (Rumelhart, 1986).



Perceptron multicouche (PMC)

Problème XOR

31

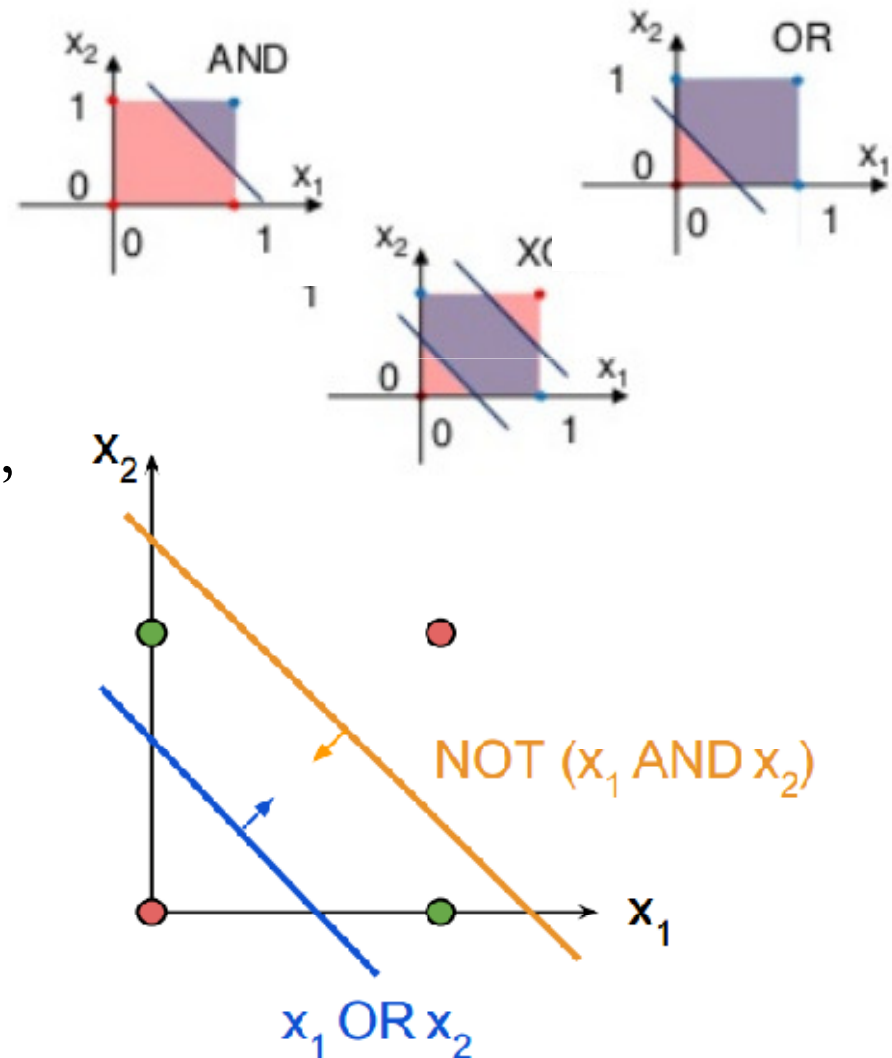


Perceptron multicouche (PMC)

Espace initial de représentation

32

- 1^{ère} vision du PMC (couche d'entrée vers couche cachée)
 - ▣ nous disposons d'une série de droites frontières définies dans l'espace initial de représentation, imbriquées de manière à produire une séparation non-linéaire

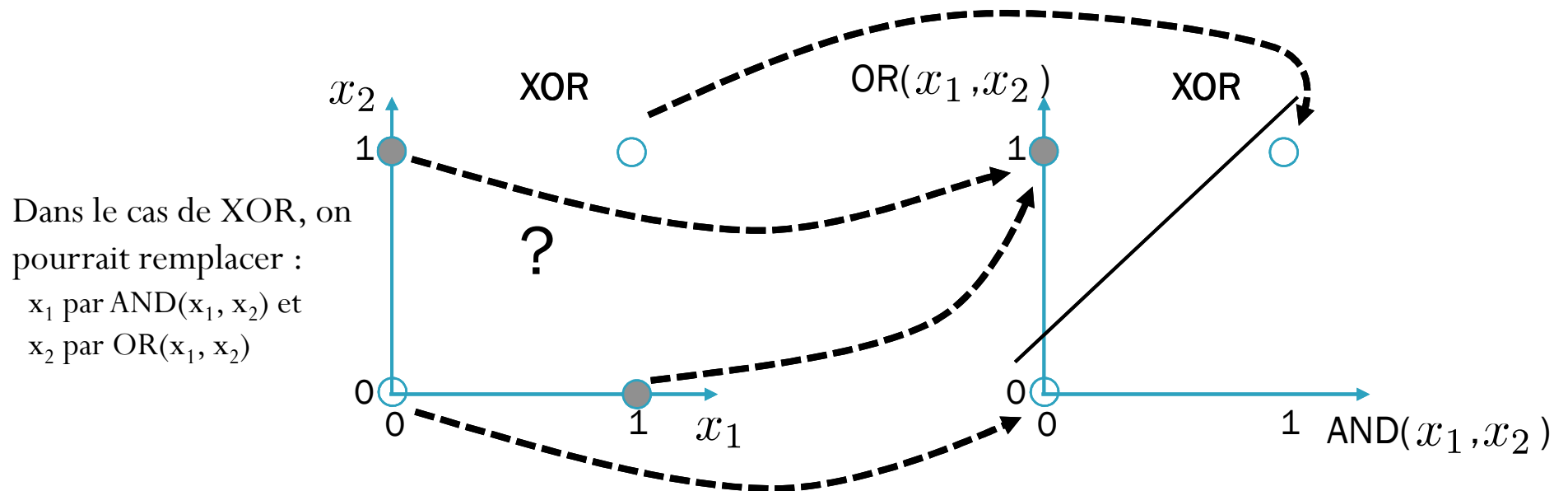


Perceptron multicouche (PMC)

Espace intermédiaire de représentation

33

- 2^{ème} vision du PMC (couche cachée vers couche de sortie)
 - la couche cachée définit un espace intermédiaire de représentation (une sorte d'espace factoriel) où l'on définit un séparateur linéaire (en espérant que les points soient linéairement séparables)
 - si ce n'est pas le cas on peut augmenter le nombre de neurones dans la couche cachée



Perceptron multicouche (PMC)

Fonctions d'activation

34

Différents types de fonctions d'activation sont utilisables, par exemple :

Fonction linéaire

Output layer pour la régression

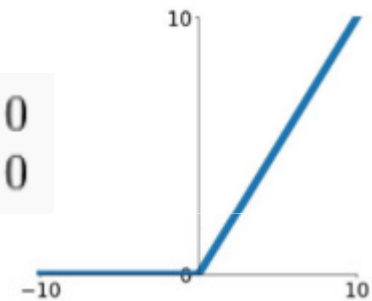
$$f(x) = x$$



Fonction ReLu (Rectified Linear units)

Filtre les valeurs négatives

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

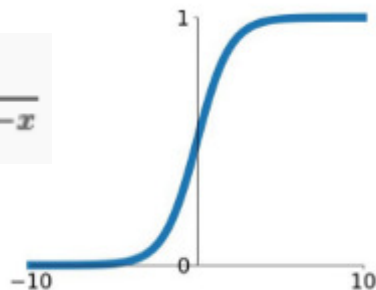


Fonction Sigmoid $\sigma(x)$

Ramène les valeurs entre $[0, 1]$.

Utilisé pour le classement. Y codé $\{0, 1\}$

$$f(x) = \frac{1}{1 + e^{-x}}$$

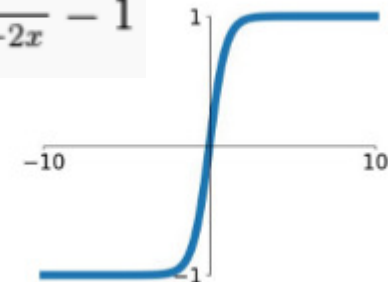


Fonction Tangente hyperbolique (tanh)

Ramène les valeurs entre $[-1, 1]$

Alternative à sigmoïde pour le classement. Y codé $\{-1, +1\}$

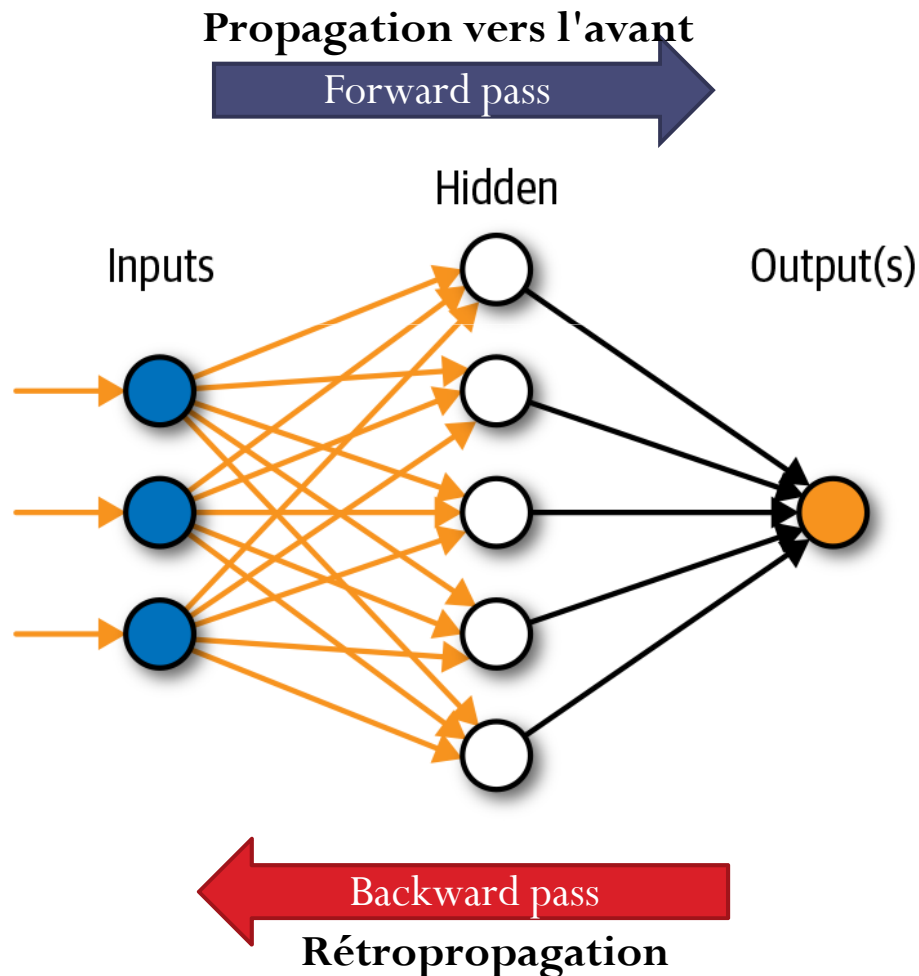
$$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$



Perceptron multicouches (PMC)

Forward et backpropagation

35



La **rétropropagation du gradient (Backpropagation)** est une méthode pour entraîner un réseau de neurones, consistant à mettre à jour les poids de chaque neurone de la dernière couche vers la première.

Elle permet de calculer le gradient de l'erreur pour chaque neurone du réseau afin de corriger les erreurs selon l'importance de la contribution de chaque élément à celles-ci.

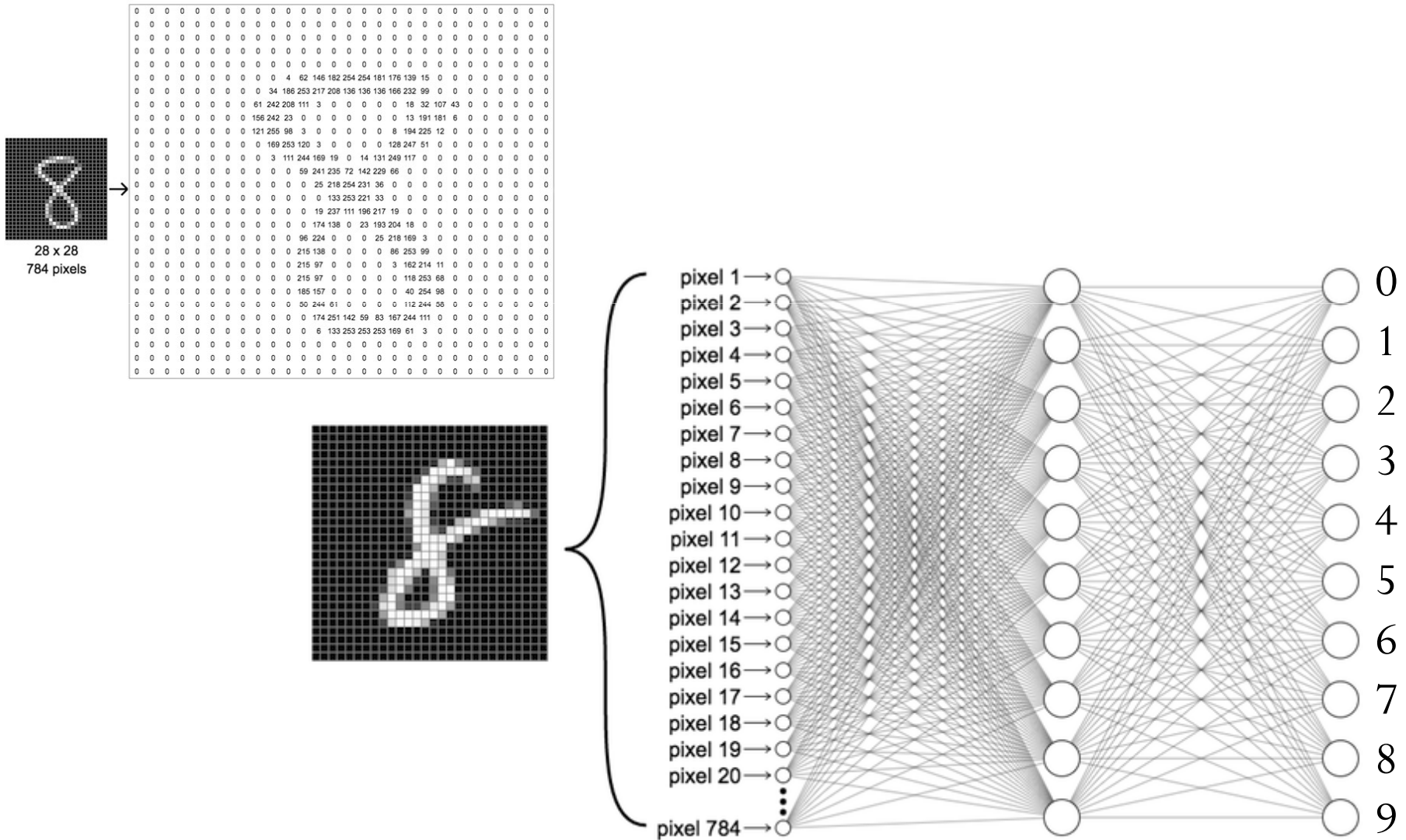
$$W_{inew} = W_{iold} - \eta * \frac{\partial E}{\partial W_i}$$

E est la fonction de perte à optimiser, par exemple : MSE ou Cross Entropy.

Perceptron multicouches (PMC)

Exemple : reconnaissance des chiffres manuscrits

36



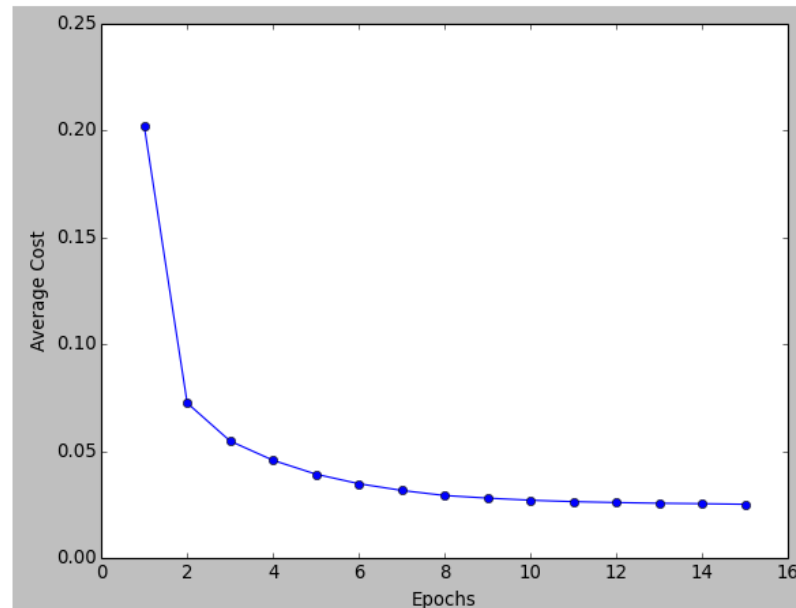
Descente de gradient

Batch, Mini batch et stochastique

37

□ Descente de Gradient classique (batch gradient descent)

- ▣ On fait passer la totalité des observations, le gradient est calculé, les coefficients sont corrigés. Etc.



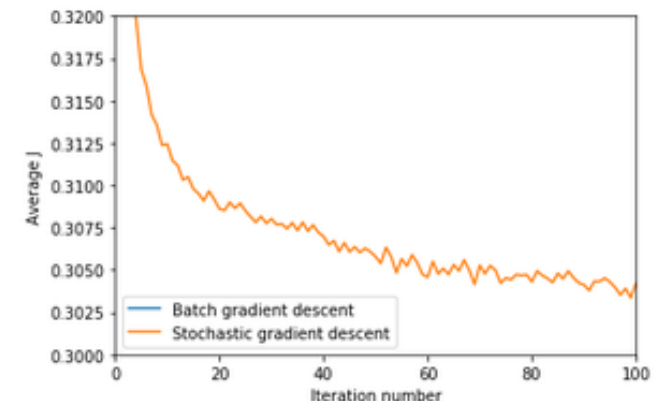
Descente de gradient

Batch, Mini batch et stochastique

38

□ Descente de Gradient stochastique (online)

- C'est est une approximation de la descente de gradient, applicable lorsque la fonction objectif s'écrit comme une somme de fonctions dérivables
 - c'est très souvent le cas en apprentissage supervisé (ou on s'arrange pour que ce soit le cas)
- Gradient calculé pour chaque observation



Descente de gradient

Batch, Mini batch et stochastique

39

□ Descente de Gradient stochastique (online)

▣ Pour chaque itération:

- Prendre un seul exemple ($x^{(i)}$) des données d'entraînement
- Entraîner le réseau de neurones sur cet exemple
- Calculer son gradient
- Utiliser le gradient calculé pour mettre à jour les poids
- Répéter les étapes précédentes pour tous les exemples des données d'entraînement

Descente de gradient

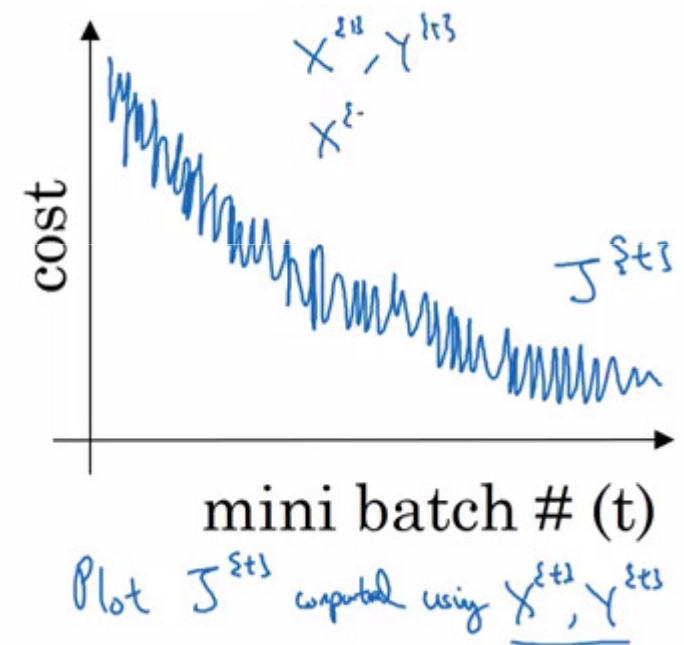
Batch, Mini batch et stochastique

40

□ Mini-batch (traitement par lots)

- ▣ On fait passer un groupe d'observations
 - effectif : paramètre de l'algorithme
- ▣ Pour chaque itération:
 - Prendre un mini-batch (des données d'entraînement)
 - Entraîner le réseau de neurones sur ce lot
 - Calculer la moyenne du gradient du mini-batch
 - Utiliser la moyenne du gradient calculé pour mettre à jour les poids
 - Répéter les étapes précédentes pour tous les lots créés

Le traitement par lots permet d'améliorer la convergence en réduisant le nombre de passages sur la base entière. Il permet également de se contenter de charger partiellement les données en mémoire au fur et à mesure.

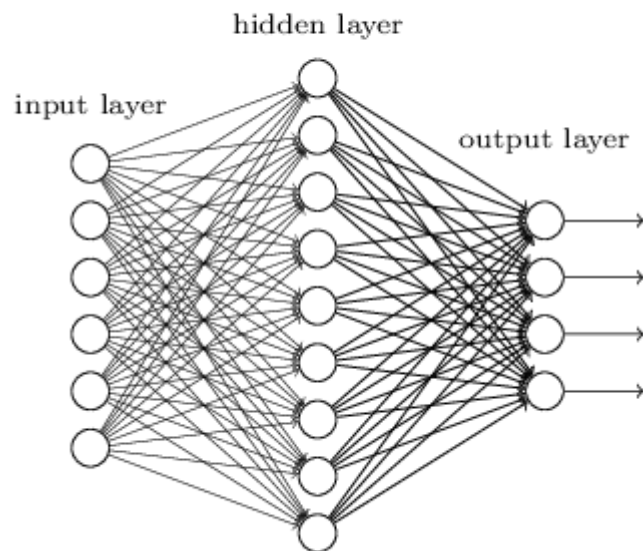


Réseaux de neurones profonds

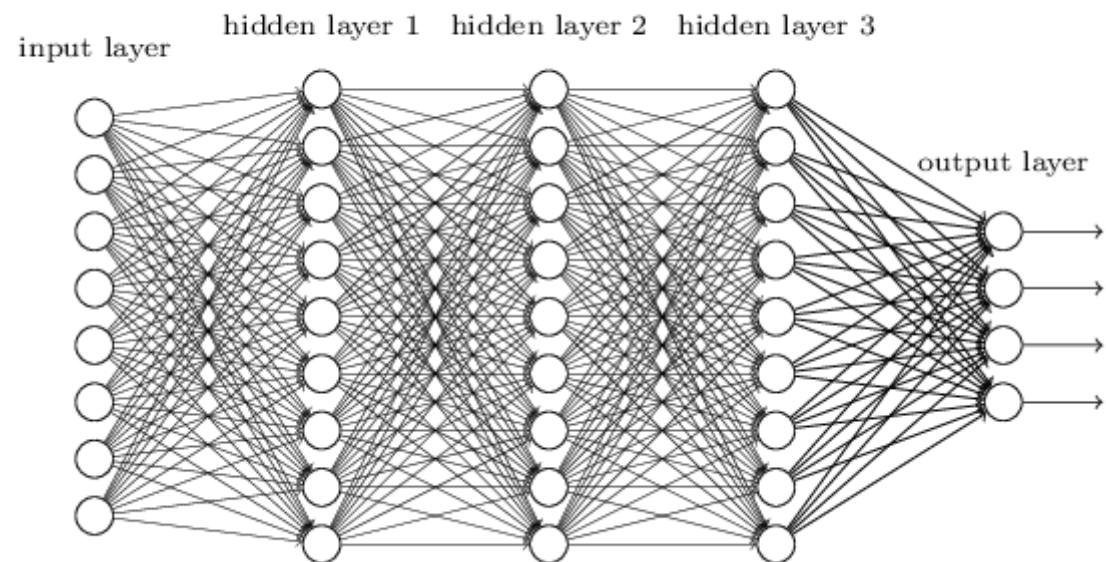
41

En 2012, lors d'une compétition organisée par ImageNet, le CNN AlexNet parvint pour la première fois à surpasser un humain (taux d'erreur plus réduit) dans la reconnaissance d'images, ce qui déclencha la ruée vers l'IA, enfin crédible aux yeux des industriels.

"Non-deep" feedforward
neural network



Deep neural network



42

Classification naïve bayésienne

Classificateur bayésien naïf

43

- La classification naïve bayésienne est un type de classification bayésienne probabiliste simple basée sur le théorème de Bayes avec une forte indépendance des hypothèses.
- Elle met en œuvre un classifieur bayésien naïf, ou classifieur naïf de Bayes, appartenant à la famille des classifieurs linéaires.

Classificateur bayésien naïf

44

Théorème de Bayes :

The diagram shows the equation $P(c | x) = \frac{P(x | c) P(c)}{P(x)}$ with four blue arrows pointing from labels to parts of the equation: 'Likelihood' points to $P(x | c)$, 'Class Prior Probability' points to $P(c)$, 'Posterior Probability' points to $P(c | x)$, and 'Predictor Prior Probability' points to $P(x)$.

$$P(c | x) = \frac{P(x | c) P(c)}{P(x)}$$

Labels and arrows:

- Likelihood (points to $P(x | c)$)
- Class Prior Probability (points to $P(c)$)
- Posterior Probability (points to $P(c | x)$)
- Predictor Prior Probability (points to $P(x)$)

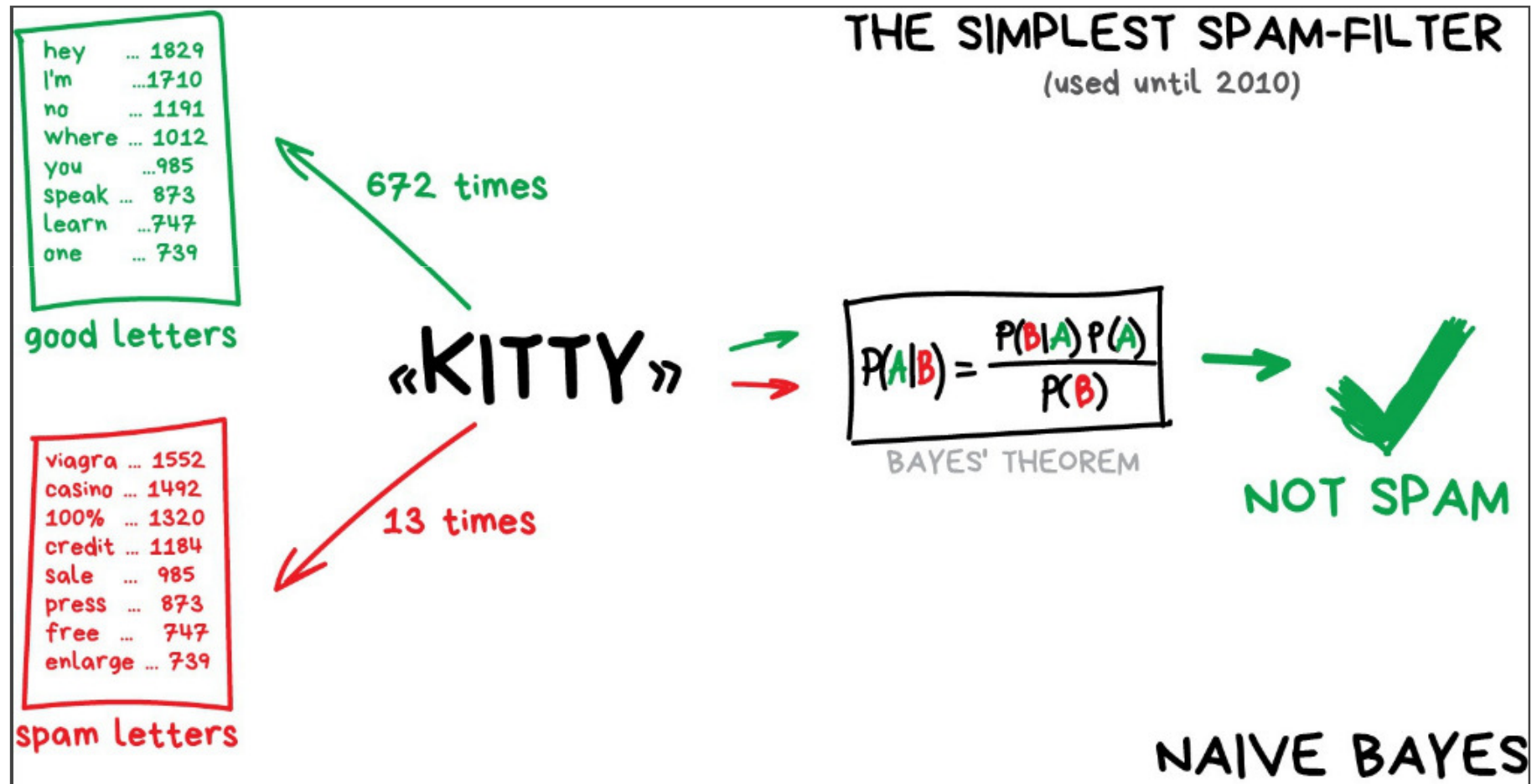
Hypothèse: indépendance des attributs

$$P(x_1, \dots, x_n | C) = P(x_1 | C) \dots P(x_n | C)$$

(Voir : complément du cours)

Applications

45



Exercice

46

Temps	Temperature	Humidite	Vent	Class
soleil	chaud	élevé	faux	N
soleil	chaud	élevé	VRAI	N
couvert	chaud	élevé	faux	P
pluie	tiede	élevé	faux	P
pluie	froid	normal	faux	P
pluie	froid	normal	VRAI	N
couvert	froid	normal	VRAI	P
soleil	tiede	élevé	faux	N
soleil	froid	normal	faux	P
pluie	tiede	normal	faux	P
soleil	tiede	normal	VRAI	P
couvert	tiede	élevé	VRAI	P
couvert	chaud	normal	faux	P
pluie	tiede	élevé	VRAI	N

Question

Classifier: $X = \langle \text{pluie, chaud, élevée, faux} \rangle$

Exercice : Correction

47

$P(P) = 9/14$
$P(N) = 5/14$

Temps	
$P(\text{soleil} P) = 2/9$	$P(\text{soleil} N) = 3/5$
$P(\text{couvert} P) = 4/9$	$P(\text{couvert} N) = 0$
$P(\text{pluie} P) = 3/9$	$P(\text{pluie} N) = 2/5$
Température	
$P(\text{chaud} P) = 2/9$	$P(\text{chaud} N) = 2/5$
$P(\text{tiède} P) = 4/9$	$P(\text{tiède} N) = 2/5$
$P(\text{froid} P) = 3/9$	$P(\text{froid} N) = 1/5$
Humidité	
$P(\text{élevée} P) = 3/9$	$P(\text{élevée} N) = 4/5$
$P(\text{normale} P) = 6/9$	$P(\text{normale} N) = 1/5$
Vent	
$P(\text{Vrai} P) = 3/9$	$P(\text{vrai} N) = 3/5$
$P(\text{faux} P) = 6/9$	$P(\text{faux} N) = 2/5$

Exercice : Correction

48

- Soit $X = \langle \text{pluie, chaud, élevée, faux} \rangle$
- $P(X | p) \cdot P(p)$
 $= P(\text{pluie} | p) \cdot P(\text{chaud} | p) \cdot P(\text{élevée} | p) \cdot P(\text{faux} | p) \cdot P(p)$
 $= 3/9 \cdot 2/9 \cdot 3/9 \cdot 6/9 \cdot 9/14 = 0.010582$
- $P(X | n) \cdot P(n)$
 $= P(\text{pluie} | n) \cdot P(\text{chaud} | n) \cdot P(\text{élevée} | n) \cdot P(\text{faux} | n) \cdot P(n)$
 $= 2/5 \cdot 2/5 \cdot 4/5 \cdot 2/5 \cdot 5/14 = 0.018286$
- X est classifié en n (ne pas jouer au tennis)